



Write Pretty

Personal Project 02

Jun Xiang

Date

Table of Contents

1. Overview	3
1.1 Problem	3
1.2 Solution	3
1.3 Goals	3
1.4 Features	4
2. Scope	5
2.1 In Scope	5
2.2 Out of Scope	5
3. Requirements	6
3.1 Functional Requirements	6
3.2 Non-Functional Requirements	6
4. Setup & Design	7
4.1 Technology Stack	7
4.2 System Architecture	7
4.3 Design Decisions	7
5. Limitations & Risks	8
6. Testings and Findings	9
6.1 Test Case 1: General Text Cleanup	9
6.2 Test Case 2: Markdown & Code Preservation	10
6.3 Test Case 3: URL & Email Handling	11
6.4 Test Case 4: History Mode	12
6.5 Test Case 5: Text Statistics, Controls & Scrolling	14
6.6 Test Case 6: Local Processing	16
7. Challenges	17
8. Improvements	18

1. Overview

WritePretty is a lightweight text cleanup tool that improves the consistency and readability of written text using simple, rule-based processing.

It can clean spacing and punctuation, correct clear capitalisation issues, and highlight possible spelling, capitalisation, and formatting problems for review.

WritePretty does not use AI to rewrite the user's text. Its purpose is to make small, predictable improvements while keeping the user's original wording.

1.1 Problem

Many users write content in plain text or Markdown but spend extra time manually fixing:

- spelling mistakes
- inconsistent capitalisation
- spacing and punctuation issues
- inconsistent text formatting

Many existing tools focus on grammar improvement or AI rewriting when only simple, predictable cleanup may be needed.

1.2 Solution

WritePretty is a lightweight text formatting and correction tool that improves text consistency and readability without using AI. The tool automatically:

- detects and highlights spelling issues
- corrects clear capitalisation and formatting issues
- highlights uncertain issues for review
- preserves the user's original wording

1.3 Goals

The goal of this project is to develop a simple and private text processing tool that helps users improve the quality and consistency of their writing through deterministic rules rather than AI-generated modifications.

The project aims to help:

- reduce time spent on manual text cleanup.
- provide immediate feedback on formatting issues.
- preserve the user's original content while applying only necessary formatting and capitalisation corrections while highlighting spelling issues for review.
- demonstrate the use of text processing and rule-based automation.

1.4 Features

Test Input	Description
Text Cleanup	Removes unnecessary spacing and fixes clear punctuation and capitalisation issues.
Spelling Detection	Highlights words that may be misspelled without changing them automatically.
Issue Highlighting	Uses different underline colours for spelling, capitalisation, and formatting issues.
Side-by-Side View	Shows the original and processed text together for easy comparison.
Markdown-Aware Processing	Preserves common Markdown structure and protects code from cleanup rules.
Protected Content	Avoids changing explicit URLs, email addresses, inline code, and code blocks.
History Mode	Stores up to five text entries and allows users to switch between or combine them.
Synchronised Scrolling	Keeps the Original and Pretty panels at approximately the same scroll position.
Text Statistics	Displays the number of words and characters in the processed text.
Copy & Download	Allows the processed text to be copied or downloaded as a text file.
Local Processing	Processes user text in the browser without sending it to an AI service.

2. Scope

2.1 In Scope

- English text input and processing
- spelling detection and highlighting
- clear capitalisation correction
- possible capitalisation issue detection
- spacing and punctuation cleanup
- formatting issue highlighting
- Markdown-aware processing
- protection of code, explicit URLs, and email addresses
- side-by-side comparison
- History Mode
- combined history preview
- synchronised scrolling
- word and character counts
- copying processed text
- downloading processed text
- local browser-based processing

2.2 Out of Scope

The current version does not provide:

- AI-based rewriting
- advanced grammar correction
- automatic spelling correction
- content generation
- plagiarism detection
- languages other than English
- cloud document storage
- user accounts

3. Requirements

3.1 Functional Requirements

- Allow users to enter or paste text
- Process text automatically as the user types
- Correct clear spacing, punctuation, and capitalisation issues
- Highlight possible spelling errors
- Highlight uncertain capitalisation and formatting issues
- Preserve paragraphs, line breaks, and document structure
- Preserve common Markdown syntax
- Protect inline code, fenced code blocks, and indented code
- Protect explicit URLs and email addresses
- Display the original and processed text side by side
- Display processed-text word and character counts
- Allow up to five History Mode entries
- Allow history entries to be viewed together
- Keep both text panels synchronised while scrolling
- Allow users to copy processed text
- Allow users to download processed text as a .txt file
- Provide clear visual indicators for detected issues:
 - Red: possible spelling issue
 - Blue: possible formatting issue
 - Yellow: possible capitalisation issue

3.2 Non-Functional Requirements

- Respond quickly for normal text inputs
- Have a simple interface
- Process user text locally
- Give the same result for the same input
- Avoid unnecessary changes to user content
- Remain easy to maintain and extend
- Work in modern web browsers

4. Setup & Design

4.1 Technology Stack

WritePretty is a client-side web application built with:

- HTML for application structure
- CSS for styling
- JavaScript for text processing and interaction
- Typo.js with a local English Hunspell dictionary for spell checking

4.2 System Architecture

- WritePretty uses a simple browser-based processing flow: User Input → Rule-Based Processing → Processed Output → User Review
- All text processing is performed in the browser and occurs as soon as the user types.

4.3 Design Decisions

- User text is processed in the browser instead of being sent to an AI rewriting service.
- Rule-based processing is used to produce predictable and consistent results.
- Only clear issues are changed automatically. Less certain issues are highlighted instead.
- Spelling mistakes are not automatically replaced because the intended word may not always be clear.
- Code, URLs, email addresses, and Markdown structure are protected where possible so cleanup rules do not accidentally change meaningful content.
- The Original and Pretty panels allow users to review changes before copying or downloading the result.

5. Limitations & Risks

WritePretty has several limitations due to its rule-based approach:

- Uncommon words, names, or technical terms may be incorrectly flagged.
- The system cannot understand sentence meaning or context as deeply as a grammar or AI writing tool.
- Complex grammar errors are outside the current scope.
- Only English is currently supported.
- Bare domains such as “example.com” may sometimes be treated like normal punctuation because they do not contain an explicit URL prefix.
- Code is protected from cleanup, which means errors inside code are intentionally left unchanged.
- Multiple consecutive line breaks are preserved instead of automatically removed.
- A single line break by itself is not treated as the start of a new sentence. Capitalisation occurs when another rule identifies a clear sentence, paragraph, or Markdown start.
- Rule-based detection can produce both false positives and false negatives.

6. Testings and Findings

6.1 Test Case 1: General Text Cleanup

Test Input	Expected Result	Findings
hello david.this sentence has a spelingg error. i am fine this line stays lowercase this new paragraph starts here.	Hello david. This sentence has a spelingg error. I am fine this line stays lowercase This new paragraph starts here.	• Clear spacing and capitalisation issues were corrected. • Spelling and uncertain capitalisation issues were highlighted. • Line and paragraph structure was preserved.

Test Evidence

20 words · 117 characters

HistoryDownloadCopy

Original

hello david.this sentence has a spelingg error. i am fine
this line stays lowercase

this new paragraph starts here.

Pretty

Hello david. This sentence has a spelingg error. I am fine
this line stays lowercase

This new paragraph starts here.

6.2 Test Case 2: Markdown & Code Preservation

Test Input	Expected Result	Findings
<pre># hello world **this is important.** - first item - second item Use `const name = "david";` here. ```js const name = "david"; let number = 5;</pre>	<pre># Hello world **This is important.** - First item - Second item Use `const name = "david";` here. ```js const name = "david"; let number = 5;</pre>	• Markdown structure was preserved. • Readable Markdown text was cleaned and capitalised. • Inline and fenced code remained unchanged. • Formatting inside protected code was not modified.

Test Evidence

27 words · 156 characters

History Download Copy

Original

```
# hello world

**this is important.**

- first item
- second item

Use `const name = "david";` here.

```js
const name = "david";
let number = 5;
```

Pretty

```
Hello world

This is important.

- First item
- Second item

Use `const name = "david";` here.

```js
const name = "david";
let number = 5;
```

6.3 Test Case 3: URL & Email Handling

Test Input	Expected Result	Findings
Visit Https://example.com/MyFile today.	Visit https://example.com/MyFile today.	• Explicit https:// URLs were protected. • www. URLs were protected. • Email addresses were preserved. • The URL scheme was normalised from Https:// to https:// • Bare domains remain a known limitation.
Go to www.example.com/help for more information.	Go to www.example.com/help for more information.	
Email test@example.com for help.	Email test@example.com for help.	
Visit example.com today.	Visit example. Com today.	

Test Evidence

17 words · 150 characters

HistoryDownloadCopy

Original

Visit Htps://example.com/MyFile today.

Go to www.example.com/help for more information.

Email test@example.com for help.

Visit example.com today.

Pretty

Visit https://example.com/MyFile today.

Go to www.example.com/help for more information.

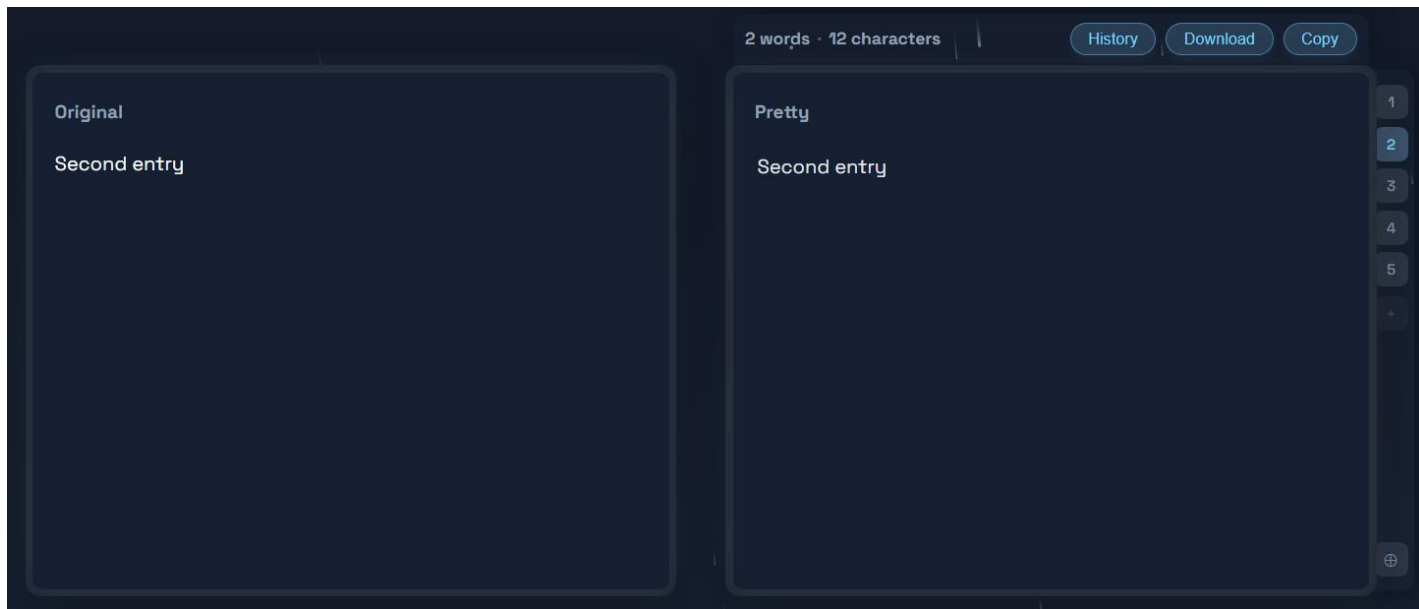
Email test@example.com for help.

Visit example. Com today.

6.4 Test Case 4: History Mode

Test Steps	Expected Result	Findings
1. Type "First entry" 2. Turn History on. 3. Confirm tab 1 appears with "First entry" 4. Click '+' once. 5. Type "Second entry" 6. Confirm tab 1 and tab 2 contain the correct text. 7. Click '+' until there are 5 entries, and confirm '+' becomes disabled. 8. Click '⊕' and confirm all entries appear together. 9. While in Combined View, delete one history entry and confirm the combined text updates while the remaining entries are still shown. 10. Click ⊕ again to return to the previously active entry.	Each history entry keeps its own text. Users can switch between entries, view them together, and delete entries. A maximum of five entries can be created.	• History entries were stored separately. • Switching tabs restored the correct text. • Combined View displayed all entries together. • Deleting an entry in Combined View updated the combined text correctly. • Returning from Combined View restored the active entry. • The '+' button was disabled after five entries.

Test Evidence

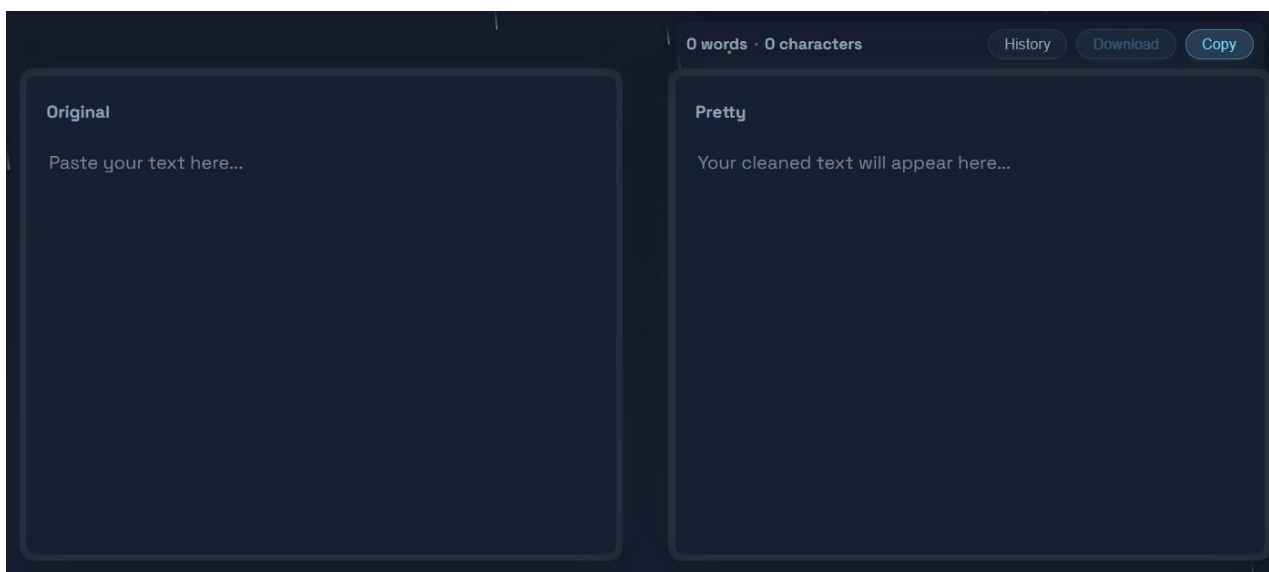
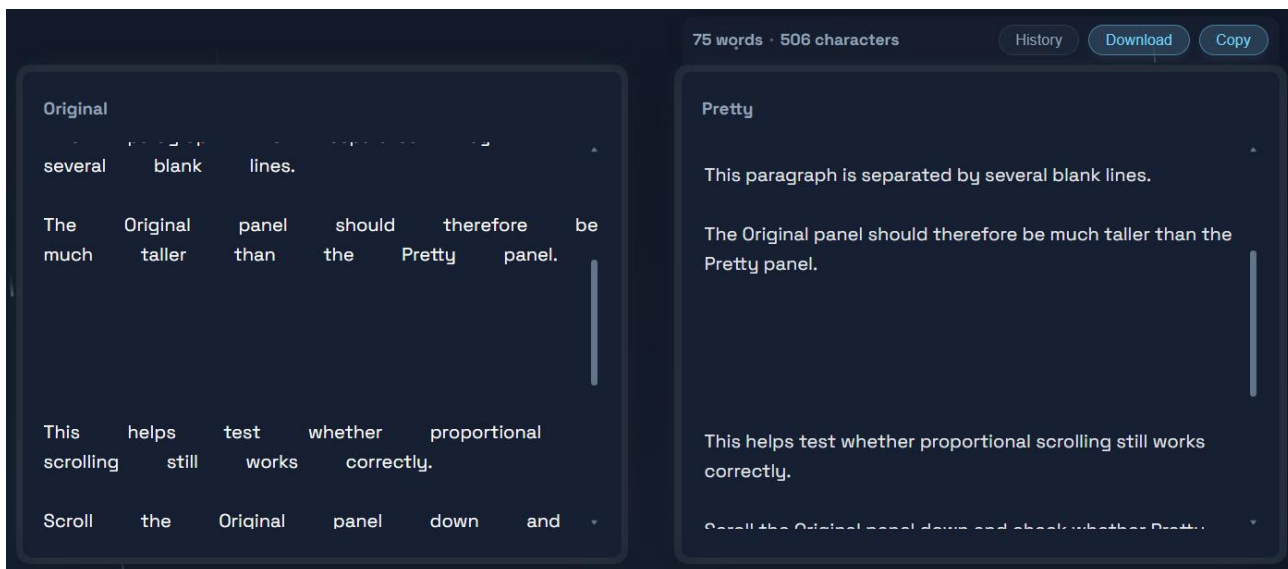
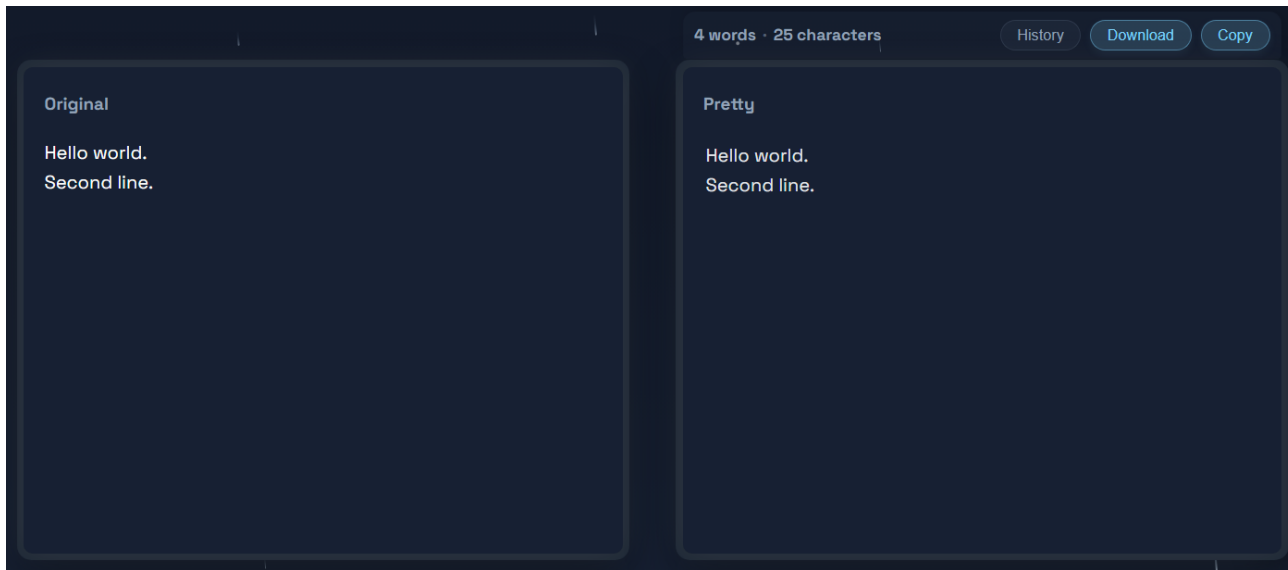




6.5 Test Case 5: Text Statistics, Controls & Scrolling

Test Input	Expected Result	Findings
1. Type "Hello world." and add a new line with "Second line." 2. Confirm the word and character counts. 3. Click Copy. 4. Click Download. 5. Clear the text and confirm Download becomes disabled. 6. Enter enough text to make both panels scrollable and scroll each panel.	The processed text shows "4 words · 25 characters." Copy and Download use the Pretty output. Download is disabled when no processed text is available. Original and Pretty scroll together proportionally.	• Word and character counts updated correctly. • Spaces, punctuation, and the line break were included in the character count. • Copy placed the processed output on the clipboard. • Download exported the processed output as a .txt file. • Download was disabled when the output was empty. • Scrolling either panel kept the other panel aligned proportionally.

Test Evidence



6.6 Test Case 6: Local Processing

Test Input	Expected Result	Findings
1. Open Developer Tools and select the Network tab. 2. Clear the existing network log. 3. Type or paste text into WritePretty. 4. Edit the text several times and use the cleanup features. 5. Check whether typing or processing creates any new requests containing the user's text.	Text cleanup should happen directly in the browser without sending the entered text to an external processing service.	• Text processing was performed locally in the browser. • Typing and editing did not send the entered text to an external text-processing service. • No AI or server-based rewriting requests containing the user's input were observed.

Test Evidence

The screenshot shows the WritePretty web application interface on the left and the browser's developer tools network tab on the right.

WritePretty Interface:

- Header: "Keep your words. Make it pretty." with a subtitle "A simple rule-based text cleanup tool that improves consistency and readability."
- Text input area: "47 words · 288 characters" with buttons for "History", "Download", and "Copy".
- Two text boxes: "Original" and "Pretty". Both contain the same five-step test procedure.

Browser Developer Tools Network Tab:

Name	Status	Type	Initiator	Size	Time	Fill...
index.html	304	docum...	Other	242 B	2.04 s	
style.css	304	stylesh...	index.html:10	243 B	2.02 s	
typo.js	304	script	index.html:161	243 B	2.05 s	
script.js	304	script	index.html:162	243 B	2.08 s	
css2?family=Space+Grotesk...	200	stylesh...	style.css:1	0 B	1 ms	(disk c...
V8mDoQDQ5KfoMM3T6r8...	200	font	css2?family=Spa...	0 B	1 ms	(disk c...
en_US-large.aff	304	fetch	script.js:46	242 B	2.03 s	
ws	101	webso...	index.html:186	0 B	Pending	
en_US-large.dic	304	fetch	script.js:49	244 B	2.05 s	

7. Challenges

Challenges	How it was handled
It can be difficult to know when capitalisation is definitely wrong.	Only clear cases are corrected automatically. Uncertain cases are highlighted instead.
Names, brands, and technical terms may not be recognised by the dictionary.	The system avoids automatically changing them when the intended result is uncertain.
A misspelled word can have several possible corrections.	Spelling errors are highlighted instead of automatically replaced.
Normal cleanup rules can accidentally change Markdown or code.	Code and other protected content are temporarily excluded from normal cleanup.
It was difficult to tell whether unusual spacing was accidental or intentional.	The system uses conservative formatting rules and leaves more complex cases for future improvement.
Bare domains such as “example.com” can look like punctuation mistakes.	Explicit URLs and email addresses are protected, while bare domains remain a known limitation.
The project scope could easily become too large.	The project was kept focused on local, rule-based text cleanup.

8. Improvements

- **Custom Terms**
 - Let users define preferred spellings or capitalisation for names, brands, and technical terms.
 - Highlight inconsistent use of those terms.
- **Spelling Suggestions**
 - Show possible replacements for spelling errors while leaving the final choice to the user.
- **Help & Guidance**
 - Explain what each underline colour means.
 - Explain which issues are corrected automatically and which are only highlighted.
 - Explain how code, URLs, and email addresses are handled.
 - Add simple tooltips to explain buttons and controls.
- **Processing Controls**
 - Allow users to choose which cleanup rules to apply.
 - Options may include removing extra line breaks, removing extra spaces, and removing Markdown formatting markers such as “#” and “*”.
- **Mobile View**
 - Improve the layout and controls for smaller screens and mobile devices.
 - Local Autosave
 - Automatically saves the current text and History Mode entries in the browser so work can be restored after refreshing or reopening the page.